

Mandatory 1 — Local Multi-LLM Coding Workflow Evaluation

Evaluation of local multi-LLM coding workflows using separately configurable local model endpoints

1. Objective & summary

This report identifies the most viable way to run a **local, multi-LLM workflow** that collaboratively produces software artifacts across architecture, implementation, testing, documentation and deployment validation — using **local model backends** and **at least two separately configurable endpoints**.

Two established, open-source agent harnesses are evaluated and compared:

- **Hermes Agent** (Nous Research) — *recommended*.
- **OpenHands** (All-Hands-AI).

The recommendation is **Hermes**: it satisfies the assignment's hard requirement of configuration-driven routing between multiple local endpoints with *first-class* provider routing, supports parallel work via subagent delegation, and is significantly lighter to run locally (no Docker).

2. Candidate #1 — Hermes Agent (Nous Research)

Overview

[Hermes Agent](#) is a general, open-source (MIT) autonomous agent. Beyond chat it provides terminal execution, file editing, code execution, browser automation, MCP integration, a skills system, persistent memory, and — most relevant here — **subagent delegation** and **provider routing**.

Setup complexity

- **Time to first working run:** low — a one-line source installer (`iex (irm https://hermes-agent.nousresearch.com/install.ps1)` on Windows), then `hermes setup` to pick a model provider.
- **Moving parts:** the Hermes CLI plus one or more Ollama/vLLM endpoints. No Docker, no database, no Node toolchain.

Capability coverage

Responsibility	Support
Architecture	Via terminal + file tools and its skills system (on-demand docs)
Tech lead / task planning	Via subagent delegation (isolated, concurrent workstreams)
Implementation ($N \geq 2$ workers)	Subagent delegation — N concurrent child agents (3 by default, configurable)
Testing & quality	Runs commands (<code>pytest</code> , linters) through the terminal tool
Documentation	Edits files and loads project context files (<code>AGENTS.md</code> , <code>CLAUDE.md</code> , <code>.hermes.md</code>)
Deployment validation	Runs scripts/commands; checkpoints + <code>/rollback</code> provide a safety net

Multi-endpoint support

Hermes has **first-class provider routing**: multiple providers configured in `~/.hermes/config.yaml` (or `hermes model`), with priority ordering, whitelists, blacklists and automatic **fallback providers**. This maps directly to the hard requirement of routing between two separate local endpoints via configuration.

Failure modes

- **Model lacks tool calling** → agent degrades to chat only (mitigate with a tool-calling model such as qwen2.5-coder:32b or gemma4:31b).
- **Context window too small** → Ollama defaults to 2048–4096 tokens; Hermes needs ~64k (OLLAMA_CONTEXT_LENGTH=64000).
- **Provider down** → automatic fallback to a backup provider.

Overall assessment

Strengths: config-driven multi-endpoint routing, concurrent subagents, light local install. **Weaknesses:** general-purpose rather than SWE-specialised; local tool-calling quality depends on the model you run.

3. Candidate #2 — OpenHands (All-Hands-AI)

Overview

[OpenHands](#) is an open-source (MIT) software-engineering agent. It runs an autonomous plan → edit → run loop inside a Docker sandbox, aimed specifically at coding tasks in a repository.

Setup complexity

- **Time to first working run:** higher — Docker image(s), an agent runtime, model provider wiring, and a capable local model.
- **Moving parts:** OpenHands backend + frontend, Docker sandbox, the LLM provider config, and the target repository.

Capability coverage

- **Architecture / planning / implementation:** strong native support (autonomous edit-and-run loop, multi-file changes).
- **Testing / documentation / deployment:** covered via the agent running commands in the sandbox, but role separation and reporting require configuration.

Multi-endpoint support

OpenHands connects to local servers (LM Studio, Ollama, vLLM, SGLang) through an OpenAI-compatible endpoint. Its **Model Routing** (SDK Router class) and **LLM Registry** allow routing different requests to different models, but the feature is explicitly "under active development"; the default is one LLM config per session, so role-specific routing across two endpoints requires a LiteLLM proxy or a custom router.

Failure modes

- **Context overflow** on large repositories (the agent loop accumulates context).
- **Tool-calling / file-op failures** in the sandbox.
- **Model quality** — small local models have "limited functionality" per the docs.

Overall assessment

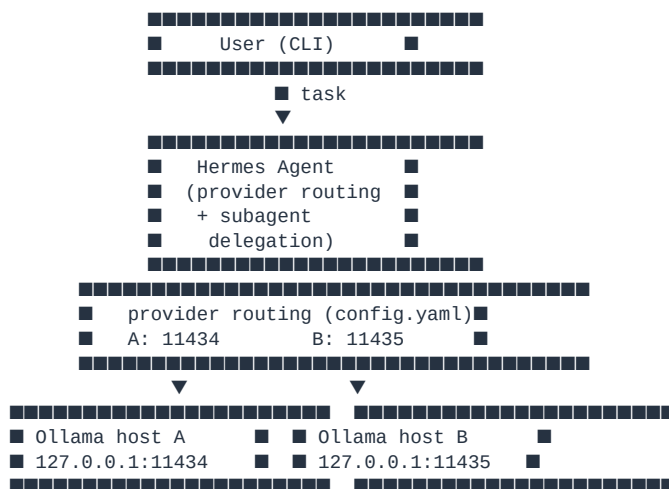
Strengths: mature, purpose-built SWE agent with real multi-file edits. **Weaknesses:** heavier Docker-based setup, and weaker first-class support for the assignment's central hard requirement of config-driven, role-specific routing between two endpoints.

4. Candidate comparison

Requirement	Hermes	OpenHands
Open source	Yes (MIT)	Yes (MIT)

Requirement	Hermes	OpenHands
Two local endpoints	Yes (multiple providers)	Yes (LM Studio/Ollama/vLLM/SGLang)
Config-driven routing	Yes — Provider Routing (priority/whitelist/blacklist)	Partial — Model Routing "under development"
Role-specific models	Yes	Limited (LiteLLM proxy / custom Router)
Parallel workers ($N \geq 2$)	Yes — subagent delegation	Yes — parallel tool execution / task tool
Architecture / planning	Yes (terminal + file + skills)	Yes (autonomous loop)
Multi-file repository changes	Yes (file editing)	Yes (native)
Testing / quality	Yes (runs commands)	Yes (sandbox commands)
Documentation	Yes (edits files)	Yes
Deployment validation	Yes (runs scripts)	Yes
Git / reviewable changes	Yes (checkpoints + /rollback)	Yes (agent commits)
Human control	Checkpoints + rollback + diff review	Sandbox confirm mode
Reproducibility	config.yaml + versioned prompts	config.toml + Docker
Security baseline	Local only	Local + Docker sandbox
Setup complexity	Low (no Docker)	High (Docker)
Overall suitability	Recommended	Comparison candidate

5. Recommended workflow architecture



A task is given to Hermes, which routes requests to the configured local endpoints (priority/whitelist/blacklist + fallback) and can spawn concurrent subagents for parallel workstreams. File edits, commands and tests run locally; checkpoints allow rolling back changes.

6. Local model endpoint architecture

Endpoint	Purpose	Host	Model	Port	Configuration
Provider A	primary (tool-calling)	localhost	qwen2.5-coder:32b	11434	base_url: http://localhost:11434/v1
Provider B	secondary / fallback	localhost	qwen2.5-coder:7b	11435	provider routing entry

Each endpoint is an ollama serve on its own port with OLLAMA_CONTEXT_LENGTH=64000. Hermes discovers endpoints via ~/.hermes/config.yaml, selects between them through **Provider Routing** and fails over automatically, and can

be remapped without changing the workflow code.

7. End-to-end demo workflow (documented reproduction)

- Install Ollama and pull a tool-calling model (qwen2.5-coder:32b).
- Start two endpoints: ollama serve on 11434 and 11435, each with OLLAMA_CONTEXT_LENGTH=64000.
- Install Hermes (iex (irm https://hermes-agent.nousresearch.com/install.ps1)).
- Configure ~/.hermes/config.yaml: provider: custom, base_url, model, and add the second endpoint under Provider Routing.
- Give Hermes a coding task (e.g. "add a login endpoint + tests") and review the resulting file diffs and checkpoints.
- Run the project tests and record results.
- Validate deployment (scripts/checklist) and update documentation.

See docs/SETUP_GUIDE.md for the exact commands.

8. Functional requirement traceability

Requirement	How Hermes satisfies it	Evidence (Hermes feature / doc)
Architecture	terminal + file tools + skills produce design artifacts	Skills system, file editing
Component decomposition	agent writes design docs	file editing
Interface contracts	agent writes OpenAPI/spec files	file editing
Deployment topology	agent documents topology/constraints	file editing
ADRs	agent writes ADR files	file editing
Tech lead task breakdown	subagent delegation partitions work	Subagent delegation
Scope / acceptance criteria / DoD	agent writes tickets	file editing
Dependency ordering	task planning via skills	Skills
Multiple coding workers ($N \geq 2$)	concurrent subagents	Subagent delegation (3 default, configurable)
Multi-file changes	agent edits multiple files	file editing
Test creation + execution	agent writes + runs tests	terminal execution
Quality report	agent runs linters/tests and reports	terminal execution
Static checks	agent runs ruff/mypy	terminal execution
Developer/user docs	agent edits README/docs	file editing
API documentation	agent writes API docs	file editing
Runbook	agent writes runbook	file editing
Design documents	agent writes design docs	file editing
Deployment validation	agent runs scripts/checklist	terminal execution

9. Non-functional requirement traceability

Requirement	Solution	Evidence
Predictability & control	checkpoints + /rollback + diff review	Checkpoints & Rollback

Requirement	Solution	Evidence
Human approval / plan + diff	review file diffs before accepting	file diffs, rollback
Git / version control	changes are ordinary repo edits	file editing
Reviewable changes	diffs + checkpoints	checkpoints
Reproducibility	config.yaml + versioned prompts	~/.hermes/config.yaml
Context management	skills (progressive disclosure), context files, subagent isolation	Skills, Context Files, Subagents
Scaling with repository size	skills load on demand; subagents isolate context	Skills system
Security baseline	local-only endpoints	Ollama loopback
No public unauthenticated endpoint	endpoints bind localhost	local-ollama guide

10. Failure modes & recovery

Failure	Detection	Recovery	Human intervention
Model lacks tool calling	tool calls degrade to text	switch to a tool-calling model	pick qwen2.5-coder:32b/gemma4:31b
Context window too small	startup rejection / truncation	OLLAMA_CONTEXT_LENGTH=64000	set env + restart
Endpoint A down	provider error	automatic fallback to provider B	restart endpoint
Subagent failure	subagent returns error	retry / reduce concurrency	review task
Invalid generated code	tests fail	agent re-runs; /rollback	review diff
File op failure	tool error	checkpoint restore	review

11. Tradeoffs

- **Specialisation vs. routing:** OpenHands is the more mature SWE agent, but its model routing is still maturing; Hermes is more general but has first-class multi-provider routing and concurrent subagents.
- **Setup complexity vs. capability:** Hermes installs without Docker; OpenHands' Docker sandbox is heavier but more isolated.
- **Model quality vs. hardware:** both need a capable local model for reliable tool calling (24GB+ VRAM recommended); small models work but degrade.
- **Automation vs. control:** both offer human review (Hermes checkpoints/rollback, OpenHands sandbox confirm mode).

12. Recommendation

Selected toolchain: Hermes Agent (Nous Research).

Hermes best satisfies the assignment's hard requirement — **two separately configurable local endpoints with configuration-driven routing** — through its first-class **Provider Routing** (priority/whitelist/blacklist) and **fallback providers**, and it covers the "multiple workers" requirement through **subagent delegation**. It is also materially easier to run locally (a single installer, no Docker), which makes the required demo reproducible with minimal friction.

OpenHands is a stronger pure software-engineering agent and a legitimate alternative, but its model routing is explicitly still under active development and its Docker-based setup is heavier — a poorer fit for this specific requirement set.